



Projet DATAImmo



Contexte



Réseau d'agence immobilière



DATAImmo



Analyser le marché



Aider les agences régionales à accompagner leurs clients

Les données initiales

- Le projet DATAImmo s'appuie sur 3 fichiers
 - Demandes de valeurs foncières (DVF) → Open Data
 - Ventes des biens immobiliers français du 1er semestre 2020
 - 34169 lignes

Date mutation	Nature mutation	Valeur foncière	No voie	B/T/Q	Code type de voie	Type de voie	Code voie	Voie	Code ID commune	Code postal	Commune	Code departement	Code commune
2020/01/02	Vente	165000	347		0	RUE	20	DU CHATEAU	1	1170	CHEVRY	01	103
2020/01/02	Vente	355680	4		15	BD	1000	EDOUARD BAUDOIN	205	6160	ANTIBES	06	004
2020/01/02	Vente	229500	20	B	0	RUE	3975	MARCEAU	142	6000	NICE	06	088

- Référentiel géographique français → data.gouv
 - 38916 lignes

reggrp_nom	reg_nom	reg_nom_ok	aca_nom	dep_nom	com_code	com_code1	com_code2	com_id	com_nom_maj_court	com_nom_maj	com_nom
Province	Auvergne-Rhône-Alpes	Rhône-Alpes	Lyon	Ain	01001	1001	1001	C01001	L'ABERGEMENT CLEMENCIAT	L'ABERGEMENT-CLEMENCIAT	L'Abergement-Clémenciat
Province	Auvergne-Rhône-Alpes	Rhône-Alpes	Lyon	Ain	01002	1002	1002	C01002	L'ABERGEMENT DE VAREY	L'ABERGEMENT-DE-VAREY	L'Abergement-de-Varey
Province	Auvergne-Rhône-Alpes	Rhône-Alpes	Lyon	Ain	01003	1003	1003	C01003	AMAREINS	AMAREINS	Amareins

- Résultats des recensements de population → INSEE
 - 34991 lignes

CODREG	CODDEP	CODARR	CODCAN	CODCOM	COM	PMUN	PCAP	PTOT
84	01	02	08	001	L'Abergement-Clémenciat	779	19	798
84	01	01	01	002	L'Abergement-de-Varey	256	1	257
84	01	01	01	004	Ambérieu-en-Bugey	14134	380	14514
84	01	02	22	005	Ambérieu-en-Dombes	1751	25	1776

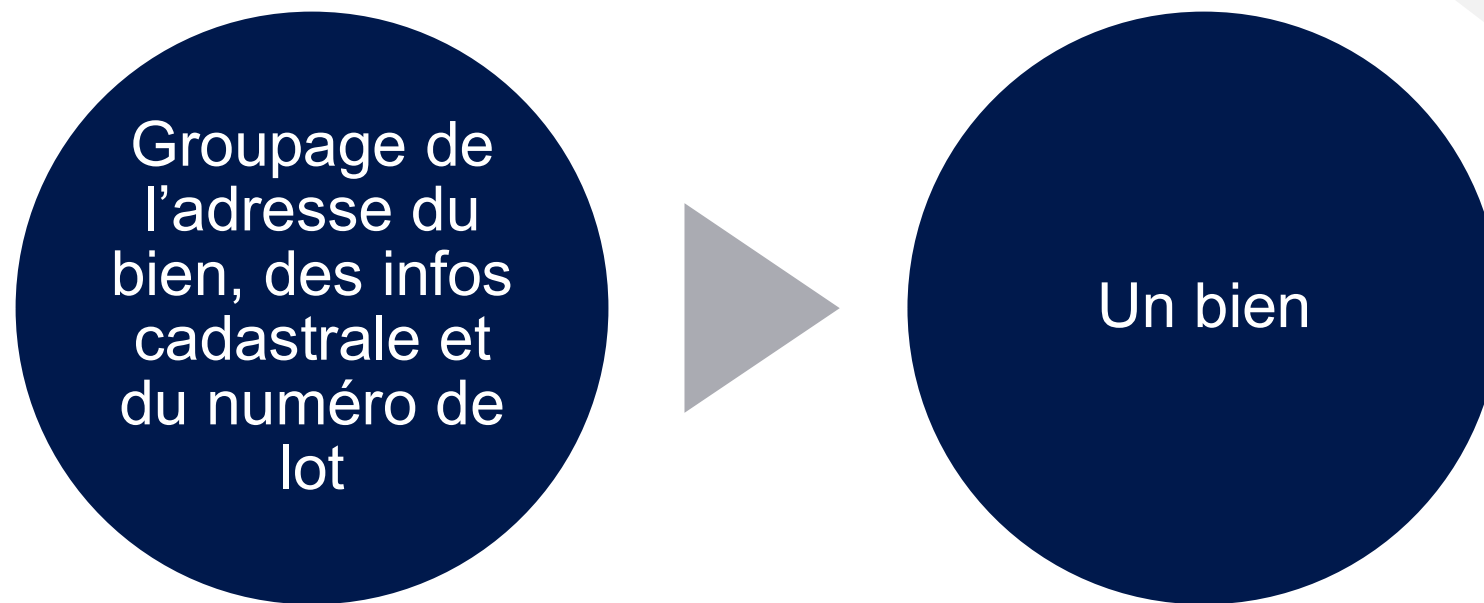


Consolidation des données



Power query

Lien entre les biens et les ventes



Inspection des données des DVF

Code postal

Le code postal n'est pas lié à une seule commune

Code postal	Commune
1090	MONTCEAUX
1100	OYONNAX
1100	OYONNAX
1100	BELLIGNAT

ANNECY
74000
74370
74600
74940
74960

Code postal
nan

Voie

Le code de la voie n'est pas unique

Code voie	Voie
1	PRINCIPALE
1	DE LA BASCULE
1	DES AJONCS
1	DES ALLUAZ
10	D ABONDANCE
10	D ADELIAU
10	ALBERT MEUNIER
10	DE BORDEAUX

Subdivisions des données des DVF

TypeLocal

- code_type_local
- type_local

TypeVoie

- code_type_voie
- type_voie

Voie

- id_voie
- code_voie
- voie

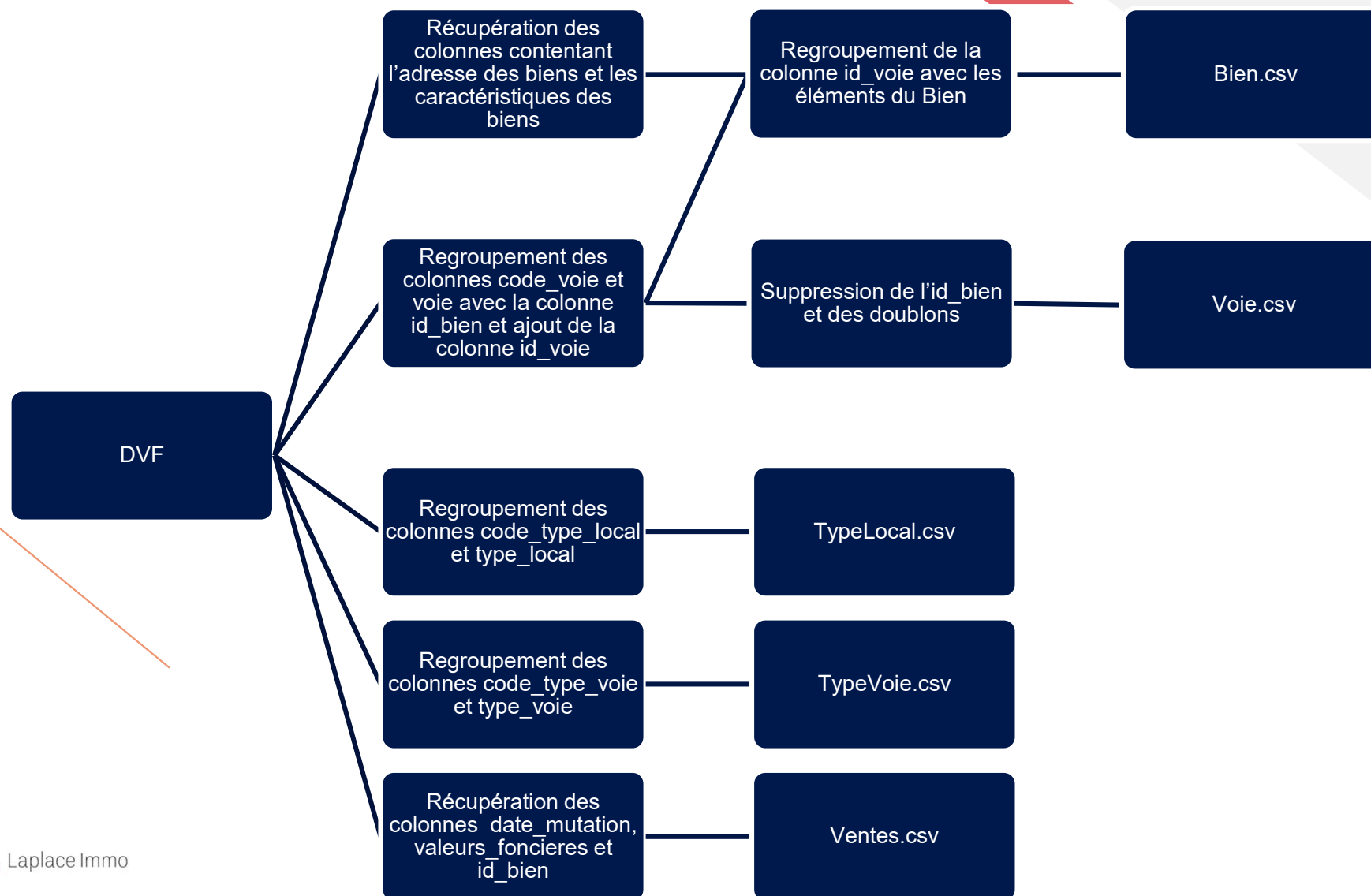
Vente

- date_mutation
- valeur_fonciere

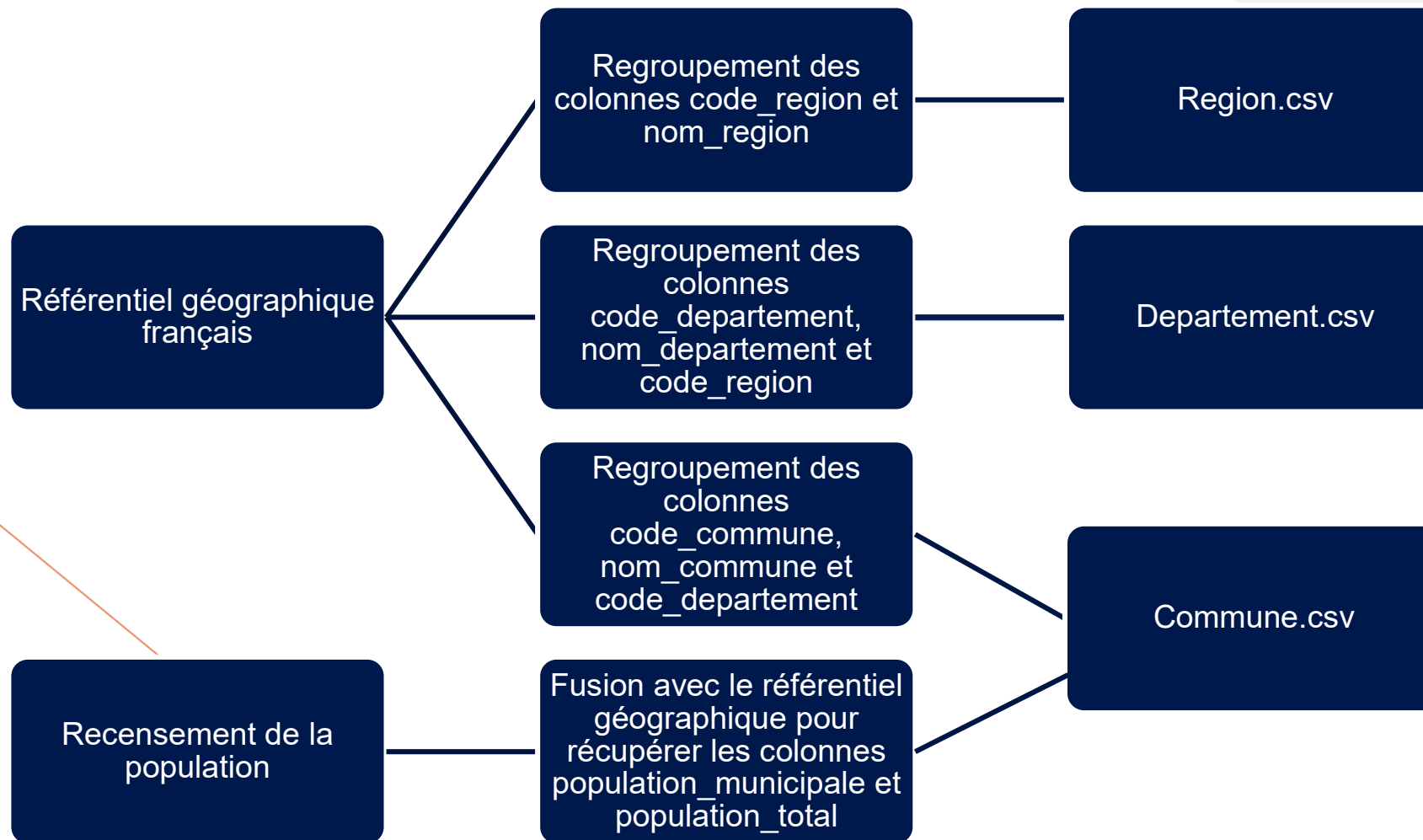
Bien

- id_bien
- num_voie
- b_t_q
- code_postal
- surface_carrez
- surface_réelle
- nb_pieces

Consolidation des données des DVF



Consolidation des données géographiques



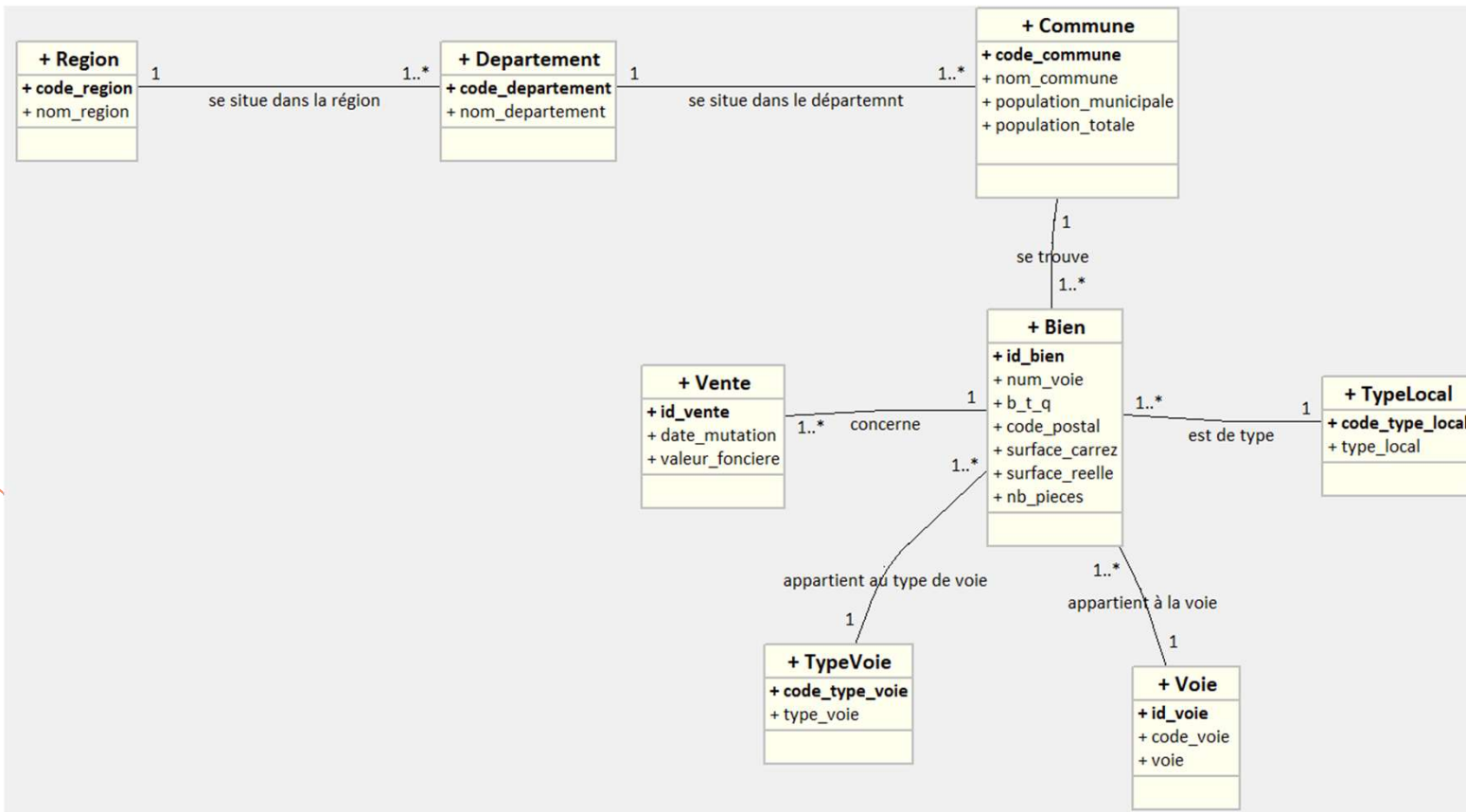
Dictionnaire des données : Ventes

CODE	SIGNIFICATION	CLE	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL	
id_bien	Identifiant du bien	Primary	Integer	NC	Elémentaire	Ne doit pas être nul Numérique généré automatiquement et incrémenté de 1	NC	Bien
num_voie	Numéro de la voie	NC	Integer	NC	Elémentaire	Peut-être nul	NC	
b_t_q	Indice de répétition	NC	Varchar	1	Elémentaire	Peut-être nul	NC	
code_type_voie	Code permettant d'identifier le type de voie	Foreign	Integer	NC	Elémentaire	Ne doit pas être nul	NC	
code_postal	Code postal de la commune	NC	Integer	4 ou 5	Elémentaire	Peut-être nul	NC	
id_voie	Identifiant de la voie	Foreign	Integer	NC	Elémentaire	Ne doit pas être nul Numérique généré automatiquement et incrémenté de 1	NC	
code_commune	Identifiant de la commune	Foreign	Varchar	5	Elémentaire	Ne doit pas être nul Colonne com_code du référentiel géographique français qui est une concaténation du code département et du code commune	NC	
surface_carrez	Surface loi Carrez	NC	Float	15 et 2 chiffres après la virgule	Elémentaire	Ne doit pas être nul	NC	
code_type_local	Code permettant d'identifier le type de local	Foreign	Integer	NC	Elémentaire	Ne doit pas être nul	NC	Vente
surface_reelle	Surface réelle du bien	NC	Float	15 et 2	Elémentaire	Ne doit pas être nul	NC	
nb_pieces	Nombre de pièces principales de l'habitation	NC	Integer	NC	Elémentaire	Ne doit pas être nul	NC	
id_vente	Identifiant de la vente	Primary	Integer	NC	Elémentaire	Ne doit pas être nul Numérique généré automatiquement et incrémenté de 1	NC	
id_bien	Identifiant du bien	Foreign	Integer	NC	Elémentaire	Ne doit pas être nul Numérique généré automatiquement et incrémenté de 1	NC	
date_mutation	Date de signature de l'acte	NC	Date	10	Elémentaire	Ne doit pas être nul Format AAAA/MM/JJ	NC	
valeur_fonciere	Prix net vendeur TVA incluse et sans les frais de notaire	NC	Float	15 et 2 chiffres après la virgule	Elémentaire	Peut-être nul	NC	

Dictionnaire des données : Référentiel géographique

CODE	SIGNIFICATION	CLE	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL	
code_region	Code de la région	Primary	Varchar	2	Elémentaire	Ne doit pas être nul	NC	Region
nom_region	Nom de la région	NC	Varchar	50	Elémentaire	Ne doit pas être nul	NC	
code_departement	Code du département	Primary	Varchar	1 ou 2 ou 3	Elémentaire	Ne doit pas être nul	NC	Departement
code_region	Code de la région	Foreign	Varchar	2	Elémentaire	Ne doit pas être nul	NC	
nom_departement	Nom du département	NC	Varchar	50	Elémentaire	Ne doit pas être nul	NC	
code_commune	Identifiant de la commune	Primary	Varchar	5	Elémentaire	Ne doit pas être nul Colonne com_code du référentiel géographique français qui est une concaténation du code département et du code commune	NC	Commune
code_departement	Code du département	Foreign	Varchar	1 ou 2 ou 3	Elémentaire	Ne doit pas être nul	NC	
nom_commune	Libellé de la commune	NC	Varchar	50	Elémentaire	Ne doit pas être nul	NC	
population_municipale	Population de la commune	NC	Integer	NC	Elémentaire	Peut-être nul	NC	
population_totale	Somme de la population de la commune et de la population comptée à part	NC	Integer	NC	Elémentaire	Peut-être nul	NC	

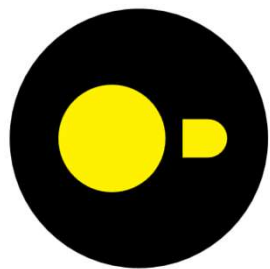
Le schéma relationnel normalisé





Base de données

Création de la base de données



DuckDB

Notebooks ▾

 data_immo

Attached databases ▾

▼  dataimmo

▼  main

 Bien

 Commune

 Departement

 Region

 TypeLocal

 TypeVoie

 Vente

 Voie

Création et remplissage de la table TypeLocal

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo;

-- Création de la table TypeLocal
CREATE TABLE IF NOT EXISTS TypeLocal(
  code_type_local INT,
  type_local VARCHAR(50) NOT NULL,
  PRIMARY KEY(code_type_local),
  CONSTRAINT type_local_chk CHECK (
    type_local in (
      'Maison',
      'Appartement',
      'Dépendance (isolée)',
      'Local industriel et commercial ou assimilés'))
);

-- Alimentation de la table TypeLocal
TRUNCATE TypeLocal;
COPY TypeLocal FROM 'C:\fichiers_sorties\TypeLocal.csv';

-- Consultation de la table TypeLocal
SELECT * FROM TypeLocal;
```

rs returned, 6 statements run in 406ms ⓘ

123	code_type_local	T	type_local
	2		Appartement
	1		Maison

Création et remplissage de la table TypeVoie

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;

-- Création de la table TypeVoie
CREATE TABLE IF NOT EXISTS TypeVoie(
    code_type_voie INT,
    type_voie VARCHAR(50),
    PRIMARY KEY(code_type_voie)
);

-- Alimentation de la table TypeVoie
TRUNCATE TypeVoie;
COPY TypeVoie FROM 'C:\fichiers_sorties\TypeVoie.csv';
```

returned, 5 statements run in 392ms ⓘ

123	Count
	80

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;
```

```
-- Consultation de la table TypeVoie
SELECT * FROM TypeVoie LIMIT 5;
```

rows returned, 3 statements run in 983ms ⓘ

123	code_type_voie	T	type_voie
	0		RUE
	15		BD
	3		RTE
	18		RES
	1		AV

Création et remplissage de la table Voie

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;

-- Création de la table Voie
DROP SEQUENCE IF EXISTS sequence_id_voie;
CREATE SEQUENCE IF NOT EXISTS sequence_id_voie INCREMENT 1;
CREATE TABLE IF NOT EXISTS Voie(
  id_voie INT DEFAULT nextval('sequence_id_voie'),
  code_voie VARCHAR(10) NOT NULL,
  voie VARCHAR(150) NOT NULL,
  PRIMARY KEY(id_voie)
);

-- Alimentation de la table Voie
TRUNCATE Voie;
COPY Voie
FROM 'C:\fichiers_sorties\Voie.csv';
```

17 rows returned, 7 statements run in 820ms ⓘ

123	Count
	22088

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;

-- Consultation de la table Voie
SELECT * FROM Voie LIMIT 10;
```

10 rows returned, 3 statements run in 546ms ⓘ

123	id_voie	T	code_voie	T	voie
	1		20		DU CHATEAU
	2		1000		EDOUARD BAUDOIN
	3		3975		MARCEAU
	4		1011		DES VESPINS RN7
	5		A084		LES ARPEGES BD DES ABA
	6		791		DU GRAND MADIER
	7		7515		DU PRADO
	8		A312		DESSUARD
	9		1090		DE NORMANDIE
	10		1037		DES NOISETIERS

Création et remplissage de la table Region

```
ATTACH IF NOT EXISTS 'dataimmo.db';  
USE dataimmo.main;
```

-- Création de la table Region

```
CREATE TABLE IF NOT EXISTS Region(  
  code_region VARCHAR(2),  
  nom_region VARCHAR(50) NOT NULL,  
  PRIMARY KEY(code_region)  
);
```

-- Alimentation de la table Region

```
TRUNCATE Region;  
COPY Region FROM 'C:\fichiers_sorties\Region.csv';
```

returned, 5 statements run in 555ms ⓘ

123	Count
	19

```
ATTACH IF NOT EXISTS 'dataimmo.db';  
USE dataimmo.main;
```

-- Consultation de la table Region

```
SELECT * FROM Region LIMIT 10;
```

rows returned, 3 statements run in 580ms ⓘ

T	code_region	T	nom_region
84			Auvergne-Rhône-Alpes
32			Hauts-de-France
93			Provence-Alpes-Côte d'Azur
44			Grand Est
76			Occitanie
28			Normandie
75			Nouvelle-Aquitaine
24			Centre-Val de Loire
27			Bourgogne-Franche-Comté
53			Bretagne

Création et remplissage de la table Département

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;

-- Création de la table Département
CREATE TABLE IF NOT EXISTS Département(
  code_département VARCHAR(3),
  nom_département VARCHAR(50) NOT NULL,
  code_region VARCHAR(2) NOT NULL,
  PRIMARY KEY(code_département),
  FOREIGN KEY(code_region) REFERENCES Region(code_region)
);

-- Alimentation de la table Département
TRUNCATE Département;
COPY Département FROM
'C:\fichiers_sorties\Département.csv';
```

rows returned, 5 statements run in 564ms ⓘ

123	Count
	109

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;
```

```
-- Consultation de la table Département
SELECT * FROM Département LIMIT 10;
```

rows returned, 3 statements run in 905ms ⓘ

T	code_département	T	nom_département	T	code_region
1			Ain		84
2			Aisne		32
3			Allier		84
4			Alpes-de-Haute-Provence		93
5			Hautes-Alpes		93
6			Alpes-Maritimes		93
7			Ardèche		84
8			Ardennes		44
9			Ariège		76
10			Aube		44

Création et remplissage de la table Commune

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;

-- Création de la table Commune
CREATE TABLE IF NOT EXISTS Commune(
  code_commune VARCHAR(5),
  nom_commune VARCHAR(50) NOT NULL,
  population_municipale INT,
  population_totale INT,
  code_departement VARCHAR(3) NOT NULL,
  PRIMARY KEY(code_commune),
  FOREIGN KEY(code_departement) REFERENCES Departement(code_departement)
);

-- Allimentation de la table Commune
TRUNCATE Commune;
COPY Commune FROM
'C:\fichiers_sorties\Commune.csv';
```

14 returned, 5 statements run in 879ms ⓘ

123	Count
	38916

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;

-- Consultation de la table Commune
SELECT * FROM Commune LIMIT 10;
```

10 rows returned, 3 statements run in 557ms ⓘ

T	code_commune	T	nom_commune	123	population_municipale	123	population_totale	T	code_departement
	01001		L'Abergement-Clémenciat		NULL		NULL		1
	01002		L'Abergement-de-Varey		NULL		NULL		1
	01003		Amareins		NULL		NULL		1
	01004		Ambérieu-en-Bugey		NULL		NULL		1
	01005		Ambérieux-en-Dombes		NULL		NULL		1
	01006		Ambléon		NULL		NULL		1
	01007		Ambronay		NULL		NULL		1
	01008		Ambutrix		NULL		NULL		1
	01009		Andert-et-Condon		NULL		NULL		1
	01010		Anglefort		NULL		NULL		1

Création de la table Bien

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;

-- Création de la table Bien
DROP SEQUENCE IF EXISTS sequence_id_bien;
CREATE SEQUENCE IF NOT EXISTS sequence_id_bien INCREMENT 1;
CREATE TABLE IF NOT EXISTS Bien(
    id_bien INT DEFAULT nextval('sequence_id_bien'),
    num_voie INT,
    b_t_q VARCHAR(1),
    code_postal INT,
    surface_carrez DECIMAL(15,2) NOT NULL,
    surface_reelle DECIMAL(15,2) NOT NULL,
    nb_pieces INT NOT NULL,
    code_type_local INT NOT NULL,
    id_voie INT NOT NULL,
    code_type_voie INT NOT NULL,
    code_commune VARCHAR(5) NOT NULL,
    PRIMARY KEY(id_bien),
    FOREIGN KEY(code_type_local) REFERENCES TypeLocal(code_type_local),
    FOREIGN KEY(id_voie) REFERENCES Voie(id_voie),
    FOREIGN KEY(code_type_voie) REFERENCES TypeVoie(code_type_voie),
    FOREIGN KEY(code_commune) REFERENCES Commune(code_commune)
);

-- Alimentation de la table Bien
TRUNCATE Bien;
COPY Bien FROM
'C:\fichiers_sorties\Bien.csv';
```

w returned, 7 statements run in 1.5s ⓘ

123	Count
	34169

Remplissage de la table Bien

```
ATTACH IF NOT EXISTS 'dataimmo.db';  
USE dataimmo.main;
```

```
-- Consultation de la table Bien  
SELECT * FROM Bien LIMIT 10;
```

123 id_bien	123 num_voie	T b_t_q	123 code_postal	.00 surface_carrez	.00 surface_reelle	123 nb_pieces	123 code_type_local	123 id_voie	123 code_type_voie	T code_commune
1	347	NULL	1170	48.22	48.00	3	2	1	0	01103
2	4	NULL	6160	39.11	40.00	1	2	2	15	06004
28790	5	NULL	69420	111.50	110.00	5	1	1	1	69007
3	20	B	6000	80.25	82.00	3	2	3	0	06088
4	550	NULL	6700	27.51	27.00	1	2	4	3	06123
5	9300	NULL	13400	47.33	47.00	2	2	5	18	13005
23536	14	NULL	6000	59.25	53.00	2	2	3	0	06088
6	27	NULL	13600	25.31	24.00	1	2	6	0	13028
7	360	NULL	13008	70.84	70.00	3	2	7	1	13208
8	5076	F	13012	67.19	66.00	3	2	8	25	13212

Création et remplissage de la table Vente

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;

-- Création de la table Vente
DROP SEQUENCE IF EXISTS sequence_id_vente;
CREATE SEQUENCE IF NOT EXISTS sequence_id_vente START 1 INCREMENT 1;
CREATE TABLE IF NOT EXISTS Vente(
    id_vente INT DEFAULT nextval('sequence_id_vente'),
    date_mutation DATE NOT NULL,
    valeur_fonciere DECIMAL(15,2),
    id_bien INT NOT NULL,
    PRIMARY KEY(id_vente),
    FOREIGN KEY(id_bien) REFERENCES Bien(id_bien)
);

-- Alimentation de la table Vente
TRUNCATE Vente;
COPY Vente (date_mutation, valeur_fonciere, id_bien) FROM
'C:\fichiers_sorties\Vente.csv';
```

7 rows returned, 7 statements run in 1.6s ⓘ

123	Count
	34169

```
ATTACH IF NOT EXISTS 'dataimmo.db';
USE dataimmo.main;
```

```
-- Consultation de la table Vente
SELECT * FROM Vente LIMIT 10;
```

10 rows returned, 3 statements run in 209ms ⓘ

	123 id_vente	📅 date_mutation	.00 valeur_fonciere	123 id_bien
1	1	2020-01-02	165000.00	1
2	2	2020-01-02	355680.00	2
3	3	2020-01-02	229500.00	3
4	4	2020-01-02	125000.00	4
5	5	2020-01-02	90000.00	5
6	6	2020-01-02	93000.00	6
7	7	2020-01-02	298100.00	7
8	8	2020-01-02	163500.00	8
9	9	2020-01-02	53000.00	9
10	10	2020-01-02	136000.00	10



Analyse du marché immobilier

Nombre total d'appartements vendus au 1er semestre 2020

```
SELECT COUNT (distinct v.id_bien) AS nb_appartement_vendu_semestre1_2020
FROM TypeLocal t
INNER JOIN Bien b ON t.code_type_local = b.code_type_local
INNER JOIN Vente v ON b.id_bien = v.id_bien
WHERE LOWER(type_local) = 'appartement'
AND EXTRACT(YEAR FROM v.date_mutation) = 2020
AND (EXTRACT(QUARTER FROM v.date_mutation) = 1 OR EXTRACT(QUARTER FROM v.date_mutation) = 2)
;
```

returned, 3 statements run in 962ms ⓘ

123	nb_appartement_vendu_semestre1_2020
-----	-------------------------------------

	31378
--	-------

Le nombre de ventes d'appartement par région pour le 1er semestre 2020

```
SELECT
  r.nom_region as nom_region,
  COUNT (DISTINCT v.id_bien) AS nb_appartement_vendu_semestre1_2020
FROM TypeLocal t
INNER JOIN Bien b ON t.code_type_local = b.code_type_local
INNER JOIN Commune c ON b.code_commune = c.code_commune
INNER JOIN Departement d ON c.code_departement = d.code_departement
INNER JOIN Region r ON d.code_region = r.code_region
INNER JOIN Vente v ON b.id_bien = v.id_bien
WHERE LOWER(t.type_local) = 'appartement'
AND EXTRACT(YEAR FROM v.date_mutation) = 2020
AND (EXTRACT(QUARTER FROM v.date_mutation) = 1 OR EXTRACT(QUARTER FROM v.date_mutation) = 2)
GROUP BY r.nom_region
ORDER BY nb_appartement_vendu_semestre1_2020 DESC
```

nom region	nb appartement vendu semestre1 2020
Ile-de-France	13995
Provence-Alpes-Côte d'Azur	3649
Auvergne-Rhône-Alpes	3253
Nouvelle-Aquitaine	1932
Occitanie	1640
Pays de la Loire	1357
Hauts-de-France	1254
Grand Est	984
Bretagne	983
Normandie	862
Centre-Val de Loire	696
Bourgogne-Franche-Comté	376
Corse	223
Martinique	94
La Réunion	44
Guyane	34
Guadeloupe	2

Proportion des ventes d'appartements par le nombre de pièces

```
SELECT
  b.nb_pieces AS nb_piece,
  CONCAT(FORMAT('{:,.4f}', COUNT (v.id_bien) / SUM(COUNT(v.id_vente)) OVER () * 100), '%')
  AS proportion_vente_appartement
FROM TypeLocal t
INNER JOIN Bien b ON t.code_type_local = b.code_type_local
INNER JOIN Vente v ON b.id_bien = v.id_bien
WHERE LOWER(t.type_local) = 'appartement'
GROUP BY b.nb_pieces
ORDER BY b.nb_pieces ASC
```

nb_piece	proportion_vente_appartement
0	0.0956%
1	21.4768%
2	31.1779%
3	28.5742%
4	14.2138%
5	3.5503%
6	0.6501%
7	0.1721%
8	0.0542%
9	0.0255%
10	0.0064%
11	0.0032%

Liste des 10 départements où le prix du mètre carré est le plus élevé

```
SELECT
  d.nom_departement as nom_departement,
  ROUND(AVG(v.valeur_fonciere / b.surface_carrez), 0) AS valeur_fonciere_moyenne_au_m2
FROM Departement d
INNER JOIN Commune c ON d.code_departement = c.code_departement
INNER JOIN Bien b ON c.code_commune = b.code_commune
INNER JOIN Vente v ON b.id_bien = v.id_bien
WHERE v.valeur_fonciere IS NOT NULL
GROUP BY d.code_departement, d.nom_departement
ORDER BY AVG(v.valeur_fonciere / b.surface_carrez) DESC
limit 10
```

nom departement	valeur fonciere moyenne au m2
Paris	12053
Hauts-de-Seine	7219
Val-de-Marne	5343
Alpes-Maritimes	4700
Haute-Savoie	4667
Seine-Saint-Denis	4345
Yvelines	4225
Rhône	4059
Corse-du-Sud	4027
Gironde	3764

Prix moyen du mètre carré d'une maison en Île-de-France

```
SELECT
  ROUND(AVG(v.valeur_fonciere / b.surface_carrez), 0)
  AS valeur_fonciere_moyenne_au_m2_en_ile_de_france
FROM TypeLocal t
INNER JOIN Bien b ON t.code_type_local = b.code_type_local
INNER JOIN Commune c ON b.code_commune = c.code_commune
INNER JOIN Departement d ON c.code_departement = d.code_departement
INNER JOIN Region r ON d.code_region = r.code_region
INNER JOIN Vente v ON b.id_bien = v.id_bien
WHERE LOWER(t.type_local) = 'maison'
AND v.valeur_fonciere IS NOT NULL
GROUP BY r.nom_region
HAVING LOWER(r.nom_region) like '%le%de%france'
;
```

returned, 3 statements run in 372ms ⓘ

#	valeur_fonciere_moyenne_au_m2_en_ile_d...
---	---

	3745
--	------

Liste des 10 appartements les plus chers avec la région et le nombre de mètres carrés

```
SELECT
  r.nom_region AS nom_region,
  ROUND(b.surface_carrez, 0) AS surface_en_m2,
  FORMAT('{:t .0f}', v.valeur_fonciere) AS valeur_fonciere
FROM TypeLocal t
INNER JOIN Bien b ON t.code_type_local = b.code_type_local
INNER JOIN Commune c ON b.code_commune = c.code_commune
INNER JOIN Departement d ON c.code_departement = d.code_departement
INNER JOIN Region r ON d.code_region = r.code_region
INNER JOIN Vente v ON b.id_bien = v.id_bien
WHERE LOWER(t.type_local) = 'appartement'
AND v.valeur_fonciere IS NOT NULL
ORDER BY v.valeur_fonciere DESC
LIMIT 10;
```

nom region	surface en m2	valeur fonciere
Ile-de-France	9	9 000 000
Ile-de-France	64	8 600 000
Ile-de-France	21	8 577 713
Ile-de-France	43	7 620 000
Ile-de-France	253	7 600 000
Ile-de-France	140	7 535 000
Ile-de-France	361	7 420 000
Ile-de-France	595	7 200 000
Ile-de-France	123	7 050 000
Ile-de-France	79	6 600 000

Taux d'évolution du nombre de ventes entre le premier et le second trimestre de 2020

```
-- CTE contenant une requête qui calcule de nombre de vente du 1er trimestres 2020
WITH nb_vente_trimestre1_2020 AS(
  SELECT COUNT(id_vente) AS nb_vente_trimestre1_2020
  FROM Vente
  WHERE EXTRACT(YEAR FROM date_mutation) = 2020
  AND EXTRACT(QUARTER FROM date_mutation) = 1
),
-- CTE contenant une requête qui calcule de nombre de vente du 2ème trimestres 2020
nb_vente_trimestre2_2020 AS(
  SELECT COUNT(id_vente) AS nb_vente_trimestre2_2020
  FROM Vente
  WHERE EXTRACT(YEAR FROM date_mutation) = 2020
  AND EXTRACT(QUARTER FROM date_mutation) = 2
)

-- Calcul du taux d'évolution des ventes entre le premier et le second trimestre 2020
SELECT
  CONCAT(
    FORMAT(
      '{:,.2f}',
      (nb_vente_trimestre2_2020 - nb_vente_trimestre1_2020) / nb_vente_trimestre1_2020 * 100
    ),
    '%'
  )
AS taux_evolution_ventes_trimestre1_trimestre2_2020
FROM nb_vente_trimestre1_2020, nb_vente_trimestre2_2020
```

taux evolution ventes trimestre1 trimestre2 2020
3.68%

Le classement des régions par rapport au prix au mètre carré des appartement de plus de 4 pièces

```
SELECT
  r.nom_region AS nom_region,
  FORMAT(
    '{:t .0f}',
    AVG(v.valeur_fonciere / b.surface_carrez)
  ) AS valeur_fonciere_moyenne_au_m2
FROM TypeLocal t
INNER JOIN Bien b ON t.code_type_local = b.code_type_local
INNER JOIN Commune c ON b.code_commune = c.code_commune
INNER JOIN Departement d ON c.code_departement = d.code_departement
INNER JOIN Region r ON d.code_region = r.code_region
INNER JOIN Vente v ON b.id_bien = v.id_bien
WHERE LOWER(t.type_local) = 'appartement'
AND b.nb_pieces > 4
AND v.valeur_fonciere IS NOT NULL
GROUP BY r.nom_region
ORDER BY AVG(v.valeur_fonciere / b.surface_carrez) DESC
```

nom_region	valeur_fonciere_moyenne_au_m2
Ile-de-France	8 770
La Réunion	3 642
Provence-Alpes-Côte d'Azur	3 588
Corse	3 105
Auvergne-Rhône-Alpes	2 891
Nouvelle-Aquitaine	2 465
Bretagne	2 412
Pays de la Loire	2 316
Hauts-de-France	2 190
Occitanie	2 097
Normandie	2 016
Grand Est	1 541
Centre-Val de Loire	1 453
Bourgogne-Franche-Comté	1 251
Martinique	573

Liste des communes ayant eu au moins 50 ventes au 1er trimestre

```
SELECT c.nom_commune, COUNT(v.id_vente) AS nb_vente
FROM Commune c
INNER JOIN Bien b ON c.code_commune = b.code_commune
INNER JOIN Vente v ON b.id_bien = v.id_bien
WHERE EXTRACT(YEAR FROM v.date_mutation) = 2020
AND EXTRACT(QUARTER FROM v.date_mutation) = 1
GROUP BY c.nom_commune
HAVING nb_vente >= 50
ORDER BY nb_vente DESC
```

nom commune	nb_vente
Paris 17e	228
Paris 15e	215
Paris 18e	209
Nice	173
Paris 11e	169
Paris 16e	165
Bordeaux	157
Paris 14e	146
Paris 20e	127
Nantes	119
Paris 19e	116
Paris 12e	110
Paris 10e	109
Paris 9e	106
Grenoble	106
Boulogne-Billancourt	99
Paris 13e	94
Paris 7e	87
Paris 6e	86
Marseille 8e	81
Asnières-sur-Seine	81
Courbevoie	80
Paris 5e	79
Paris 3e	79

Toulouse	78
Antibes	77
Marseille 4e	72
Marseille 1er	71
Vincennes	68
Rueil-Malmaison	68
Lille	67
Marseille 9e	66
Montreuil	65
Angers	64
Nîmes	63
La Ciotat	62
Sète	62
Paris 8e	62
Rennes	61
Paris 2e	61
Paris 4e	60
Toulon	59
Levallois-Perret	59
Saint-Maur-des-Fossés	56
Versailles	54
Ajaccio	54
Puteaux	53
Issy-les-Moulineaux	50

Différence en pourcentage du prix au mètre carré entre un appartement de 2 pièces et un appartement de 3 pièces

```
--CTE contenant la moyenne du prix au m2 d'un appartement 2 pièces
WITH prix_m2_appartement_2_pieces AS(
  SELECT AVG (v.valeur_fonciere / b.surface_carrez) AS valeur_fonciere_m2_appartement_2_pieces
  FROM TypeLocal t
  INNER JOIN Bien b ON t.code_type_local = b.code_type_local
  INNER JOIN Vente v ON b.id_bien = v.id_bien
  WHERE LOWER(t.type_local) = 'appartement'
  AND b.nb_pieces = 2
  AND v.valeur_fonciere IS NOT NULL
)
```

```
--CTE contenant la moyenne du prix au m2 d'un appartement 3 pièces
, prix_m2_appartement_3_pieces AS(
  SELECT AVG (v.valeur_fonciere / b.surface_carrez) AS valeur_fonciere_m2_appartement_3_pieces
  FROM TypeLocal t
  INNER JOIN Bien b ON t.code_type_local = b.code_type_local
  INNER JOIN Vente v ON b.id_bien = v.id_bien
  WHERE LOWER(t.type_local) = 'appartement'
  AND b.nb_pieces = 3
  AND v.valeur_fonciere IS NOT NULL
)
```

différence pourcentage prix m2 appartement 2 pieces et appartement 3 pieces
12.40%

```
-- Calcul de la différence en pourcentage du prix au mètre carré entre un appartement de 2 pièces et un appartement de 3 pièces.
SELECT
  CONCAT(
    FORMAT(
      '{:,.2f}',
      (valeur_fonciere_m2_appartement_2_pieces - valeur_fonciere_m2_appartement_3_pieces)
      / valeur_fonciere_m2_appartement_2_pieces * 100
    ),
    '%'
  )
AS difference_pourcentage_prix_m2_appartement_2_pieces_et_appartement_3_pieces
FROM prix_m2_appartement_2_pieces, prix_m2_appartement_3_pieces
```

Les moyennes de valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69

```
WITH rang_commune_departement_6_13_33_59_69 AS(
  SELECT
    c.code_departement,
    c.nom_commune,
    AVG(v.valeur_fonciere) AS moyenne_valeur_fonciere,
    ROW_NUMBER() OVER (PARTITION BY c.code_departement
                        ORDER BY moyenne_valeur_fonciere DESC)
                        AS rang_commune_dans_departement
  FROM Commune c
  INNER JOIN Bien b ON c.code_commune = b.code_commune
  INNER JOIN Vente v ON b.id_bien = v.id_bien
  WHERE code_departement IN ('6', '13', '33', '59', '69')
  AND v.valeur_fonciere IS NOT NULL
  GROUP BY (c.code_departement, c.nom_commune)
)
```

```
-- Récupération des rang inférieur ou égale à 3 du CTE ci-dessus
SELECT
  code_departement,
  nom_commune,
  FORMAT('{:t .0f}', moyenne_valeur_fonciere) AS moyenne_valeur_fonciere,
  rang_commune_dans_departement
FROM rang_commune_departement_6_13_33_59_69
WHERE rang_commune_dans_departement <= 3
order by CAST(code_departement AS INT), rang_commune_dans_departement;
```

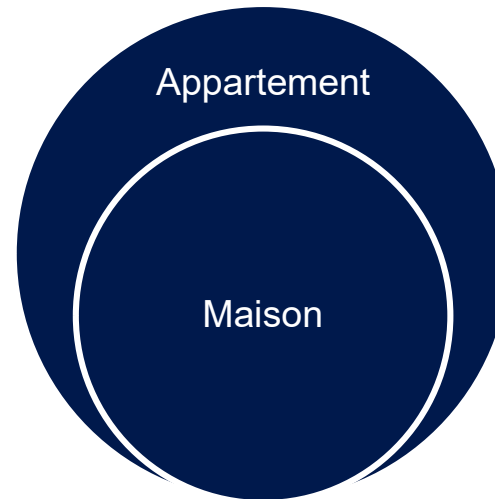
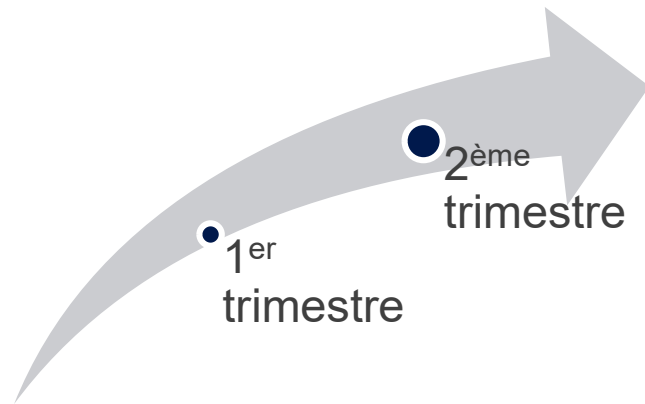
code departement	nom commune	moyenne valeur fonciere	rang commune dans departement
6	Saint-Jean-Cap-Ferrat	968 750	1
6	Eze	655 000	2
6	Mouans-Sartoux	476 898	3
13	Gignac-la-Nerthe	330 000	1
13	Saint-Savournin	314 425	2
13	Cassis	313 417	3
33	Lège-Cap-Ferret	549 501	1
33	Vayres	335 000	2
33	Arcachon	307 436	3
59	Bersée	433 202	1
59	Cysoing	408 550	2
59	Halluin	322 250	3
69	Ville-sur-Jarnioux	485 300	1
69	Lyon 2e	455 217	2
69	Lyon 6e	426 968	3

Les 20 communes avec le plus de transactions pour 1000 habitants pour les communes qui dépassent les 10 000 habitants

```
SELECT c.nom_commune,  
       FORMAT(  
         '{:,.2f}',  
         count(v.id_vente) / c.population_municipale * 1000  
       ) AS nb_transaction_pour_1000_habitants  
FROM Commune c  
INNER JOIN Bien b ON c.code_commune = b.code_commune  
INNER JOIN Vente v ON b.id_bien = v.id_bien  
WHERE c.population_municipale > 10000  
GROUP BY c.nom_commune, c.population_municipale  
ORDER BY nb_transaction_pour_1000_habitants DESC  
LIMIT 20
```

nom commune	nb transaction pour 1000 habitants
Paris 2e	5.88
Paris 1er	4.96
La Baule-Escoublac	4.74
Paris 3e	4.73
Arcachon	4.73
Paris 4e	4.12
Paris 8e	3.90
Sanary-sur-Mer	3.55
La Londe-les-Maures	3.48
Paris 9e	3.47
Paris 6e	3.45
Saint-Cyr-sur-Mer	3.31
Chantilly	3.20
Pornichet	3.14
Saint-Mandé	3.08
Paris 10e	3.05
Saint-Hilaire-de-Riez	2.92
Vincennes	2.83
Paris 17e	2.82
Enghien-les-Bains	2.79

Marché immobilier du 1^{er} semestre 2020



2 pièces

3 pièces

12 %



Merci !